

72.75: Aprendizaje Automático (Machine Learning)

Horario: Miércoles 16-19h- virtual sincrónico

Profesorado de la asignatura:

Francisco Sammartino, Ing. | frperez@itba.edu.ar

Carlos Bibián Nogueras, PhD | cbibian@itba.edu.ar

Plan de la sesión de hoy

- Aprendizaje Basado en Instancias
- knn
- weighted-knn
- ANN
- knn, DT y RF como regresores

... en la sesión anterior

- Árboles de decisión
- Hiperparámetros
- Random Forests

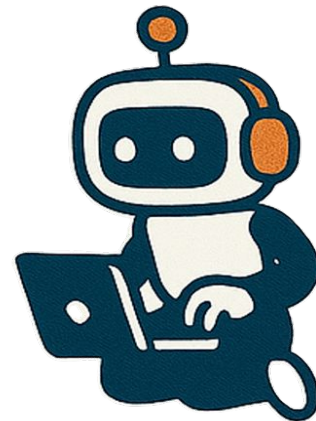
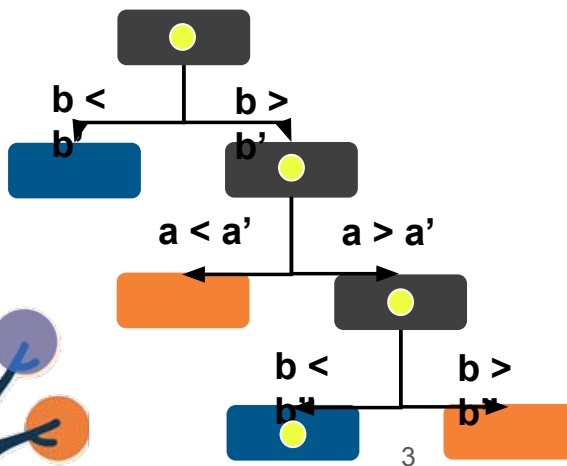
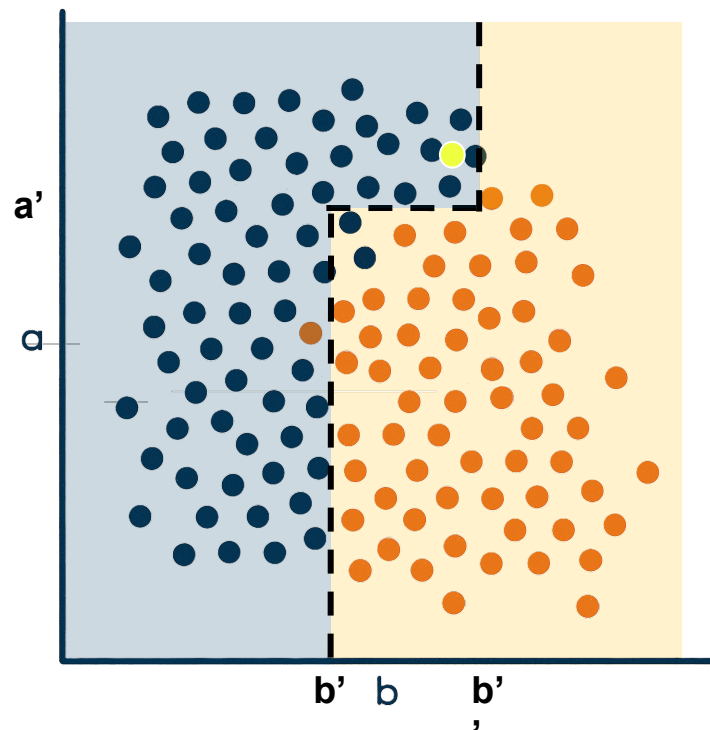
Resumen

Carlos Bibián Nogueras, PhD
cbibian@itba.edu.ar |

Instituto Tecnológico de Buenos Aires

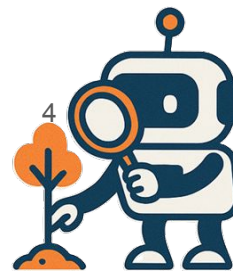
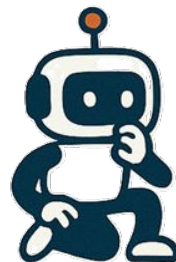
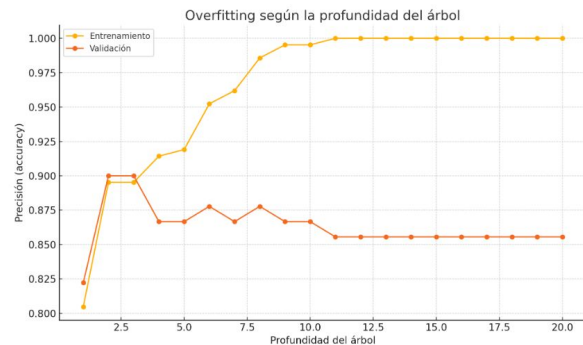
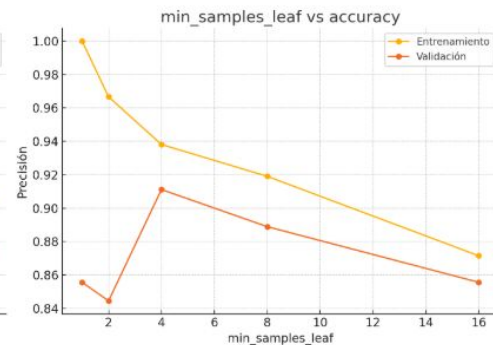
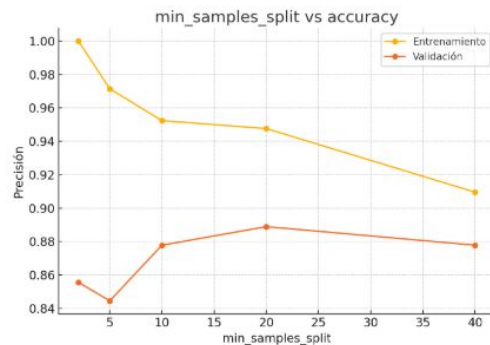
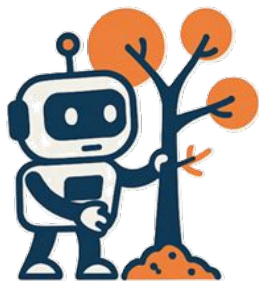
¿Cómo predice un Árbol de Decisión

How does a Decision Tree predict?



Ajuste de hiperparámetros

Hyperparameter Tunning



DT vs RF vs ET

Comparison



Decision Tree



Random Forest



Extra Trees

Selección de
feature

Todas las features
disponibles

**Subconjunto aleatorio
de features**

**Subconjunto aleatorio de
features**

Selección de
umbral

Mejor umbral (óptimo)

Mejor umbral (óptimo)

**Evalúa un umbral aleatorio
por cada feature**

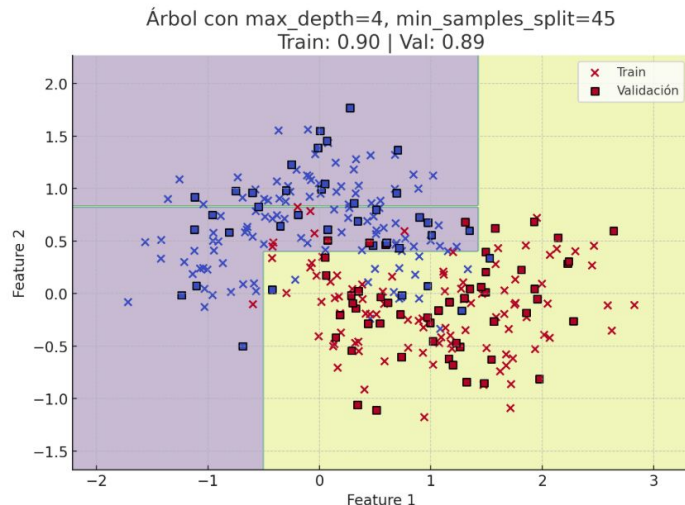
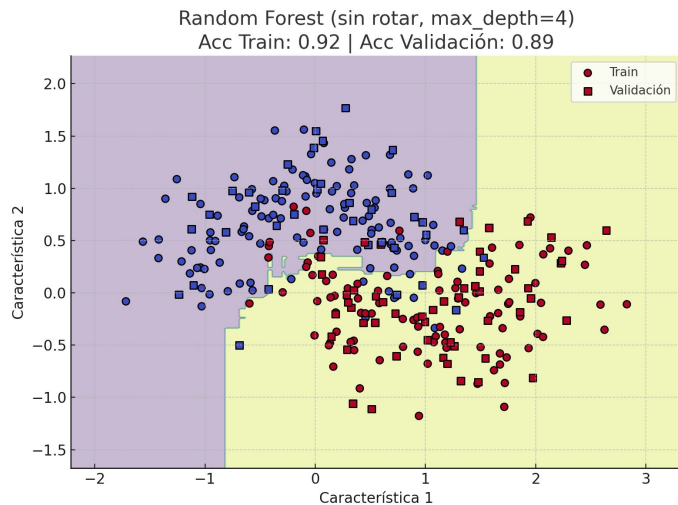
¿Qué optimiza? Mejor split global

Mejor split dentro de
features elegidas

Mejor split entre esos
candidatos aleatorios

Ejemplo

Example



****Estamos usando solo dos características así que los random forests pierden un poco de potencial en este ejemplo**

Métodos basados en vecinos

Carlos Bibián Nogueras, PhD
cbibian@itba.edu.ar |

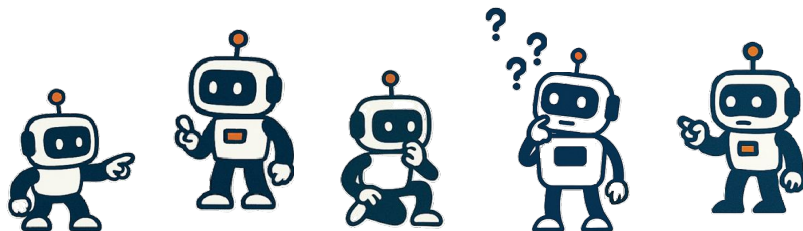
Instituto Tecnológico de Buenos Aires

Métodos basados en vecinos

Neighbor-based methods

Basar su estimación/clasificación en las **propiedades de los valores cercanos**

*Make predictions based on the **characteristics of the nearest neighbors***



Idea general formalizada

Formalizing the general idea

Suponemos **continuidad local** en nuestros datos

*We assume **local continuity** in our data.*



Instancias **cercanas** en el espacio de características deberían tener **propiedades similares**
***Nearby** instances in the feature space are expected to have **similar properties**.*

Aprendizaje perezoso

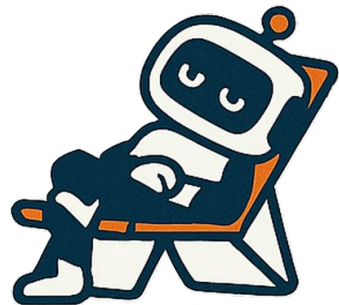
Lazy Learning

NO construyen un modelo explícito a partir de los datos

*They **do NOT build** an explicit **model** from the training data.*

Adoptan una estrategia de aprendizaje “perezosa” (**lazy learning**)

*Follow a **lazy learning** approach: learning is deferred until a prediction is needed.*



Aprendizaje perezoso

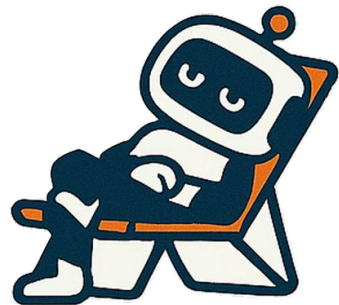
Lazy Learning

NO construyen un modelo explícito a partir de los datos

They do NOT build an explicit model from the training data.

Adoptan una estrategia de aprendizaje “perezosa” (**lazy learning**)

Follow a lazy learning approach: learning is deferred until a prediction is needed.

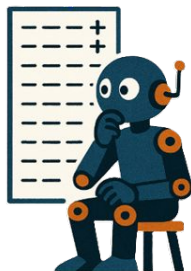


- No aprenden de los datos, sino que posponen todo el computo para el momento de la predicción
Rather than learning an explicit model from the data, they defer most of the computation until prediction time.

- Ante un dato nuevo, el algoritmo consulta los datos de entrenamiento para identificar instancias cercanas e inferir el output
Given a new data point, the algorithm searches the training set for its nearest neighbors and uses them to infer the output.

Aprendizaje Basado en Instancias vs Modelo

Instance vs Model - Based Learning



Instance-Based Learning

Literalmente “aprender de memoria”

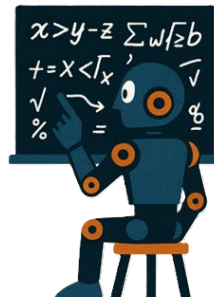
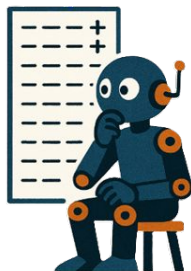
Se usan los datos para predecir – cuál y cómo es el dato más parecido

Necesitamos definir cómo se evalúa esa similitud entre datos

Model-Based Learning

Aprendizaje Basado en Instancias vs Modelo

Instance vs Model - Based Learning



Instance-Based Learning

Literalmente “aprender de memoria”

Se usan los datos para predecir – cuál y cómo es el dato más parecido

Necesitamos definir cómo se evalúa esa similitud entre datos

Model-Based Learning

A partir de los datos, crea un modelo que los representa

Es el modelo lo que se usa para generalizar/predecir para datos nuevos

Necesitamos definir cómo construir un modelo a partir de datos

Aprendizaje Basado en Instancias

Instance-Based Learning

Implicaciones / Implications

- Fase de **entrenamiento prácticamente inexistente** (lazy learning)

Virtually no training phase (lazy learning).

- La **complejidad** se traslada a la **etapa de inferencia/predicción**

Computational complexity shifts to the inference/prediction stage.

- La calidad de la predicción **depende mucho de los datos** (y la correcta elección de hiperparámetros y métricas)

Prediction quality strongly depends on the training data, as well as the choice of hyperparameters and distance metrics.



k-vecinos más cercanos

k-nearest neighbours (knn)

Carlos Bibián Nogueras, PhD

cbibian@itba.edu.ar |

Instituto Tecnológico de Buenos Aires

K-vecinos más cercanos (knn)

k-nearest neighbours (knn)

- Algoritmo más característico de esta familia.

The most representative algorithm in this family.

- Dada una nueva instancia, **identifica los k vecinos más cercanos** en el conjunto de entrenamiento y le asigna la **clase más frecuente entre esos** vecinos.

Given a new instance, it identifies the k nearest neighbors in the training set and assigns the most frequent class among them.

K-vecinos más cercanos (knn)

k-nearest neighbours (knn)

- Algoritmo más característico de esta familia.

The most representative algorithm in this family.

- Dada una nueva instancia, **identifica los k vecinos más cercanos** en el conjunto de entrenamiento y le asigna la **clase más frecuente entre esos** vecinos.

Given a new instance, it identifies the k nearest neighbors in the training set and assigns the most frequent class among them.

- K es el número de vecinos a considerar (**¡hiperparámetro!**): cuántos vecinos mirar.

K is the number of neighbors to consider (a hyperparameter!): how many neighbors to look at.

- Simple, pero muy competitivo y **capaz de captar estructuras complejas**.

Simple yet highly competitive, with the ability to capture complex patterns in the data.

Implementación

Implementation

1. Calcular la **distancia desde la nueva instancia a cada uno de los datos de entrenamiento**.

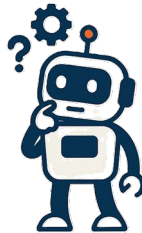
Calculate the distance between the new instance and each training data point.

2. **Ordenar las instancias** de entrenamiento según su distancia y seleccionar las **k más cercanas**.

Sort the training instances by distance and select the k nearest neighbors.

3. Asignar al nuevo punto la **clase más frecuente** entre esos k vecinos.

Assign the new data point the most frequent class among its k nearest neighbors.



¿Vecinos más cercanos?

Nearest neighbors?

¿Cómo identificamos los vecinos más cercanos? / *How do we identify the nearest neighbors?*

Dada una nueva instancia, **calculamos** su **distancia a cada instancia de entrenamiento**

Given a new instance, we compute its distance to each training instance.

Los k vecinos más cercanos son las **k instancias de entrenamiento con las distancias más pequeñas.**

The k nearest neighbors are the k training instances with the smallest distances.

¿Más cercanos? - Cálculo de distancias

Nearest? - Distances calculation

Debemos definir una **métrica de distancia**

Ésta va a determinar la geometría del espacio de características y, por lo tanto, determina también **qué instancias se consideran vecinas**.

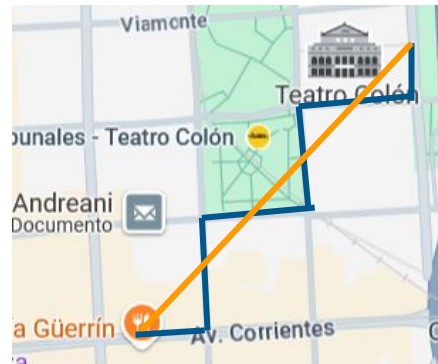
The distance metric determines the geometry of the feature space and, consequently, which instances are considered neighbors.

Euclidean distance

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

Manhattan distance

$$d(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^n |x_i - y_i|$$



¿Más cercanos? - Cálculo de distancias

Nearest? - Distances calculation

Debemos definir una **métrica de distancia**

Ésta va a determinar la geometría del espacio de características y, por lo tanto, determina también **qué instancias se consideran vecinas**.

The distance metric determines the geometry of the feature space and, consequently, which instances are considered neighbors.

Euclidean distance

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

Manhattan distance

$$d(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^n |x_i - y_i|$$

Cosine similarity

$$\cos(\theta) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|}$$

¿Más cercanos? - Cálculo de distancias

Nearest? - Distance calculation

Qué métrica de distancia utilizo? -> Depende de en qué espacio viven nuestros datos

Which distance metric should I use? -> Depends on the space our data lives in

¿Más cercanos? - Cálculo de distancias

Nearest? - Distance calculation

Qué métrica de distancia utilizo? -> Depende de en qué espacio viven nuestros datos

Which distance metric should I use? -> Depends on the space our data lives in

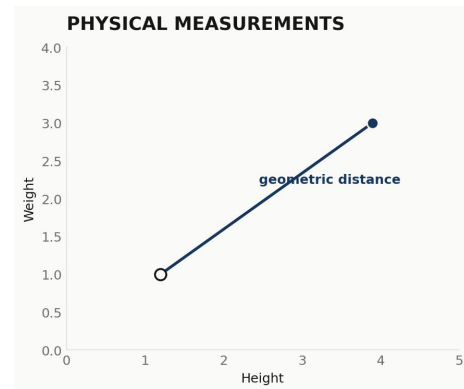
Ejemplos:

Distancia Euclidea /Euclidean distance

Medidas físicas / *Physical measurements*

-> Útil cuando las dimensiones tienen una interpretación geométrica comparable.

Useful when dimensions have a comparable geometric interpretation.



¿Más cercanos? - Cálculo de distancias

Nearest? - Distance calculation

Qué métrica de distancia utilizo? -> Depende de en qué espacio viven nuestros datos

Which distance metric should I use? -> Depends on the space our data lives in

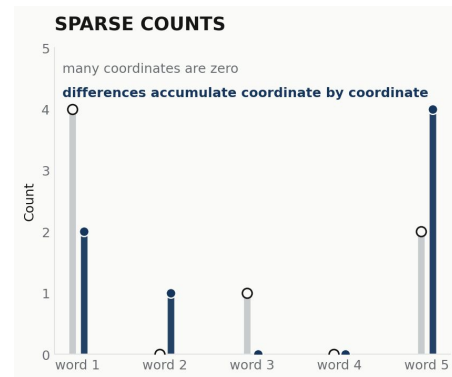
Ejemplos:

Manhattan distance

Datos dispersos/conteos / Sparse data/counts

-> Útil cuando interesa acumular diferencias coordenada por coordenada.

Useful when differences should accumulate independently across coordinates.



¿Más cercanos? - Cálculo de distancias

Nearest? - Distance calculation

Qué métrica de distancia utilizo? -> Depende de en qué espacio viven nuestros datos

Which distance metric should I use? -> Depends on the space our data lives in

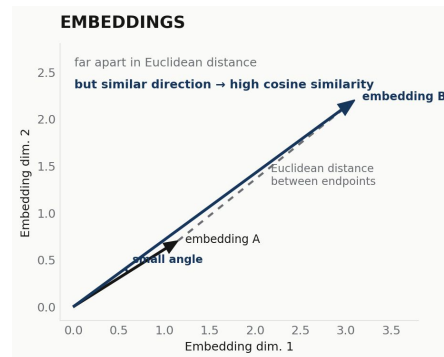
Ejemplos:

Cosine similarity

Embeddings

-> Útil cuando importa más la dirección o el patrón del vector que su magnitud.

Useful when vector direction or pattern matters more than magnitude.



¿Más cercanos? - Elección de distancias

Nearest? - Choosing distances

La métrica de distancia puede tratarse como un hiperparámetro del modelo.

The distance metric can be treated as another model hyperparameter.

¿Más cercanos? - Elección de distancias

Nearest? - Choosing distances

La métrica de distancia puede tratarse como un hiperparámetro del modelo.

The distance metric can be treated as another model hyperparameter.

También podemos transformar el espacio de features para que la distancia Euclídea sea más informativa.

We can also transform the feature space so that Euclidean distance becomes more informative.

Ejemplo: una variable cíclica como “día del año” puede codificarse con seno y coseno.

Example: a cyclic variable such as “day of the year” can be encoded using sine and cosine.

Así, valores cercanos en el ciclo también quedan cercanos bajo distancia Euclídea.

This way, values that are close in the cycle also become close under Euclidean distance.

A veces no necesitamos cambiar la distancia: necesitamos una mejor representación de los datos.

Sometimes we do not need a different distance: we need a better representation of the data.

Elección del número de vecinos (k)

Choosing the number of neighbours (k)

Principal hiperparámetro de k -NN.

The main hyperparameter of k -NN.

¿Que efecto tiene k ?

Elección del número de vecinos (k)

Choosing the number of neighbours (k)

Principal hiperparámetro de k -NN.

The main hyperparameter of k -NN.

¿Que efecto tiene k ?

- Valores **pequeños: bajo sesgo, pero alta varianza** (mayor riesgo de sobreajuste).

Small values of k : low bias but high variance (greater risk of overfitting).

- Valores **altos**: mayor estabilidad, pero alto sesgo y **menor sensibilidad a estructuras locales**.

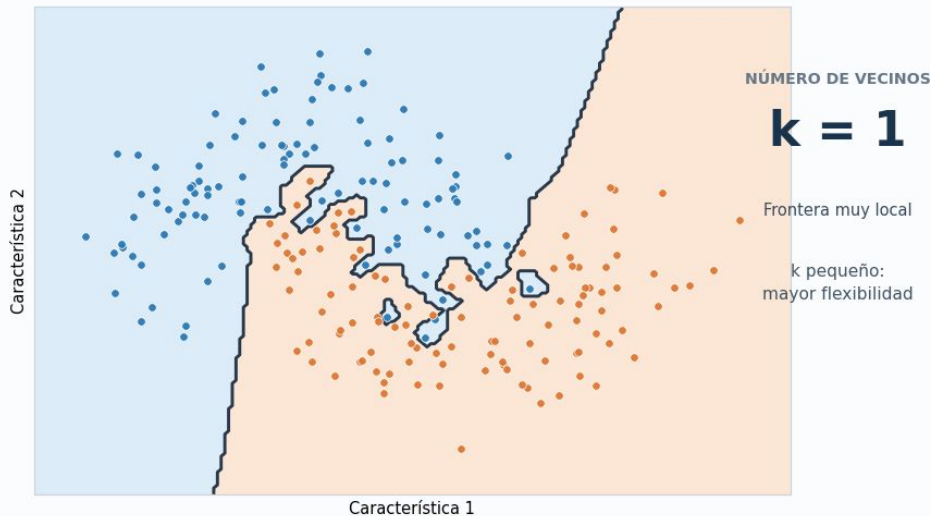
Large values of k : greater stability, but higher bias and reduced sensitivity to local patterns.

Elección del número de vecinos (k)

Choosing the number of neighbours (k)

Cómo cambia la frontera de decisión al aumentar k

Mismos datos · k -NN · voto uniforme · distancia euclídea



Elección del número de vecinos (k)

Choosing the number of neighbours (k)

Enfoque **similar al de cualquier hiperparámetro** (validación cruzada).

As with any other hyperparameter, k can be selected using cross-validation.

Consideraciones adicionales: / *Additional considerations:*

- Clases desbalanceadas: un **k alto** puede **favorecer** sistemáticamente a la **clase mayoritaria**.

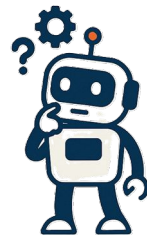
Imbalanced classes: a large k may systematically favor the majority class.

- **Dimensionalidad alta:** el concepto de **vecindad pierde fuerza**. Puede ser conveniente ajustar k junto con estrategias de reducción de dimensionalidad.

High dimensionality: the notion of proximity becomes less meaningful. Consider tuning k alongside dimensionality reduction techniques.

- **Valores impares** de k para reducir la posibilidad de empates (en problemas multiclase, evitar múltiplos del número de clases puede ayudar, aunque no garantiza evitar empates).

Odd values of k help reduce the likelihood of ties (in multiclass problems, avoiding multiples of the number of classes may help, but does not guarantee tie-free predictions).



Cálculo de distancias y escalado

Distance Calculation and Scaling

Como kNN se basa en el cálculo de distancias, es **fundamental** que las **features** estén en **escalas comparables**.
Since kNN relies on distance calculations, it is essential that features are on comparable scales.

De lo contrario, las **features con valores de mayor magnitud dominarán la distancia** y tendrán una influencia desproporcionada sobre la predicción.

Otherwise, features with larger numerical scales will dominate the distance and have a disproportionate influence on the prediction.

Normalización Min-Max

$$x = \frac{x - \min(x)}{\max(x) - \min(x)}$$

Estandarización (z-score)

$$x = \frac{x - \text{media}(x)}{\text{desviacion estandard}(x)}$$

Ponderación de vecinos

weighted k-nearest neighbours (weighted knn)

Carlos Bibián Nogueras, PhD

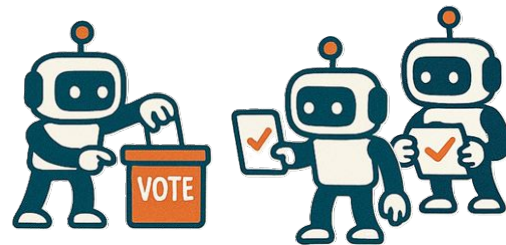
cbibian@itba.edu.ar |

Instituto Tecnológico de Buenos Aires

Ponderación de vecinos

Weighted neighbours

- El knn convencional da la misma importancia a todos los vecinos dentro del rango k, sin importar cómo de cerca estén
- Ponderación basada en distancia: a la hora de calcular la clase más presente, cada vecino tiene el **peso de la inversa de su distancia** al punto

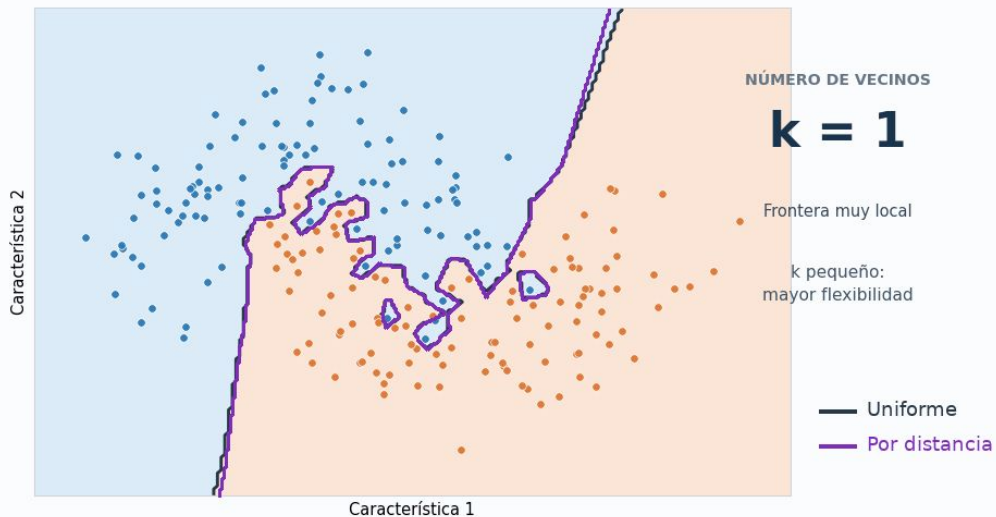


Efecto del número de vecinos (k) - ponderado

Impact of the number of neighbours (k) - weighted

Cómo cambia la frontera de decisión al aumentar k

Mismos datos · distancia euclídea · dos formas de votar



Efecto del número de vecinos (k) - ponderado

Impact of the number of neighbours (k) - weighted

Tanto **reducir k** como **ponderar**, dan **más importancia** a los vecinos **más cercanos**

¡No son equivalentes!

- Una **k muy pequeña** es más **inestable** y susceptible al ruido o al efecto de outliers
- Una k moderada (10-20), combinado con una ponderación, suele ser un sistema ser más estable y robusto

Esta **combinación** es **especialmente útil** en:

- Regiones de **alta variabilidad** como las cercanas a las **fronteras de decisión**
- Espacios con **densidad heterogénea**
- **Datos ruidosos**

Limitaciones de los métodos basados en vecinos

Limitations of Neighbor-based methods

¿Limitaciones?

Limitaciones de los métodos basados en vecinos

Limitations of Neighbor-based methods

1- Coste computacional elevado, especialmente en el momento de la predicción.

High computational cost, especially at prediction time.

- Para **clasificar** o estimar un nuevo punto, kNN debe **calcular su distancia a todos los puntos** del conjunto de entrenamiento.

To classify or predict a new point, kNN must compute its distance to every point in the training set.

Las **distancias pierden capacidad discriminativa en espacios de alta dimensionalidad**.

Distances become less discriminative in high-dimensional spaces.

- A medida que **aumenta el número de dimensiones**, las **distancias** entre puntos tienden a volverse **más similares** y la noción de “vecino cercano” pierde significado.
As dimensionality increases, distances between points tend to become more similar, and the notion of a “nearest neighbor” becomes less meaningful.
- Además, **todas** las features **contribuyen al cálculo de la distancia**, incluso aquellas poco informativas.
In addition, all features contribute to the distance calculation, including weakly informative ones.

Limitaciones de los métodos basados en vecinos

Limitations of Neighbor-based methods

2-Las **distancias pierden capacidad discriminativa en espacios de alta dimensionalidad.**

Distances become less discriminative in high-dimensional spaces.

- A medida que **aumenta el número de dimensiones**, las **distancias** entre puntos tienden a volverse **más similares** y la noción de “vecino cercano” pierde significado.
As dimensionality increases, distances between points tend to become more similar, and the notion of a “nearest neighbor” becomes less meaningful.
- Además, **todas** las features **contribuyen al cálculo de la distancia**, incluso aquellas poco informativas.
In addition, all features contribute to the distance calculation, including weakly informative ones.

Este problema suele afectar menos a los métodos basados en árboles, ya que cada split evalúa únicamente una feature cada vez.

This issue is usually less severe for tree-based methods, since each split evaluates only one feature at a time.

Limitaciones de los métodos basados en vecinos

Limitations of Neighbor-based methods

¿Soluciones?

Eficiencia en Knn

ANN, Ball Trees y ANNOY

Carlos Bibián Noguerras, PhD

cbibian@itba.edu.ar |

Instituto Tecnológico de Buenos Aires

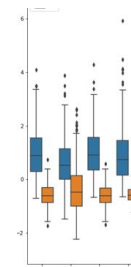
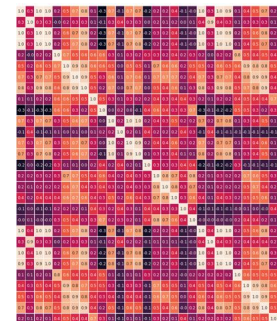
Eficiencia en KNN - Reducción de dimensionalidad

KNN Efficiency - Dimensionality Reduction

Para reducir coste computacional, pero **también para que el cálculo de distancias sea más informativo**, es recomendable reducir las dimensiones (o asegurarnos que todas aportan información relevante)

Como ya vimos en la clase 3, podemos:

- Estudiar la separabilidad entre clases que aporta cada feature (dimensión) y seleccionar aquellas con mayor valor.
- Evaluar correlación entre features
- Aplicar métodos como PCA o Autoencoders para proyectar nuestras características a un nuevo espacio de características



Eficiencia en KNN - Vecinos más cercanos aproximados (ANN)

KNN Efficiency - Approximate Nearest Neighbours (ANN)

- La **familia de métodos más usados** para **calcular vecinos** de forma **más eficiente** computacionalmente son los Approximate Nearest Neighbours (ANN)
- Encuentra vecinos cercanos de forma **aproximada**, en lugar de exacta, pero mucho más rápido (especialmente en espacios de alta dimensión o con grandes volúmenes de datos).

Eficiencia en KNN - Vecinos más cercanos aproximados (ANN)

KNN Efficiency - Approximate Nearest Neighbours (ANN)

- La **familia de métodos más usados** para **calcular vecinos** de forma **más eficiente** computacionalmente son los Approximate Nearest Neighbours (ANN)
- Encuentra vecinos cercanos de forma **aproximada**, en lugar de exacta, pero mucho más rápido (especialmente en espacios de alta dimensión o con grandes volúmenes de datos).

Funcionamiento del ANN:

- 1) Preprocesa los datos para crear una estructura que **divide el espacio de características en subespacios**.
- 2) Calcula **distancias únicamente** con los vecinos que pertenecen al mismo **subespacio** que el punto a evaluar.

Sacrifica un poco de precisión en los vecinos, **a cambio de muchísima velocidad**.

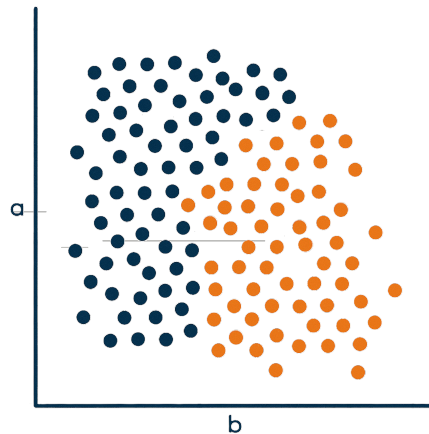
Búsqueda Informada - KD-Trees

Informed Search - KD-Trees

Utiliza **árboles de decisión** (DT) **para dividir** el espacio de características

En cada **nodo** divide el subespacio en 2 partes en base al **valor mediano en una dimensión** (la dimensión utilizada cambia en cada nodo)

Espacio dividido en regiones rectangulares que contienen un n inferior



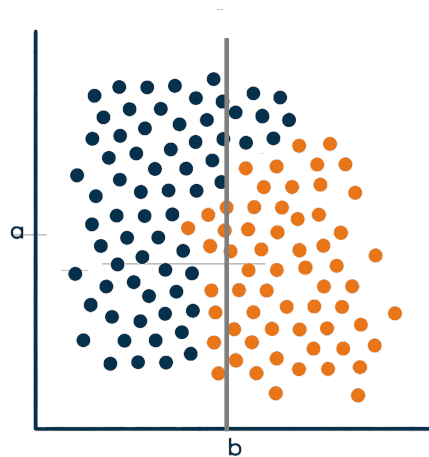
Búsqueda Informada - KD-Trees

Informed Search - KD-Trees

Utiliza **árboles de decisión** (DT) **para dividir** el espacio de características

En cada **nodo** divide el subespacio en 2 partes en base al **valor mediano en una dimensión** (la dimensión utilizada cambia en cada nodo)

Espacio dividido en regiones rectangulares que contienen un n inferior



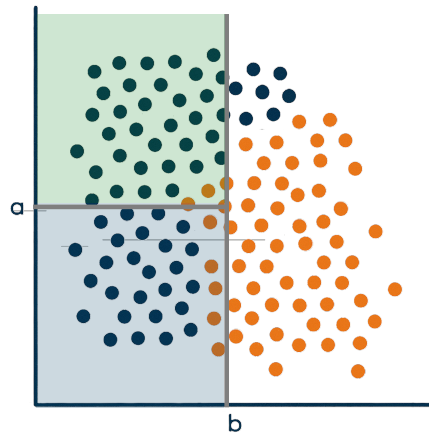
Búsqueda Informada - KD-Trees

Informed Search - KD-Trees

Utiliza **árboles de decisión** (DT) **para dividir** el espacio de características

En cada **nodo** divide el subespacio en 2 partes en base al **valor mediano en una dimensión** (la dimensión utilizada cambia en cada nodo)

Espacio dividido en regiones rectangulares que contienen un n inferior



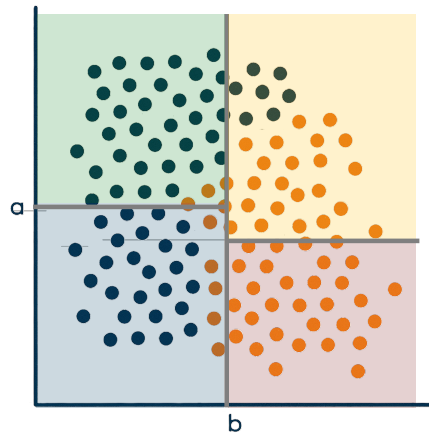
Búsqueda Informada - KD-Trees

Informed Search - KD-Trees

Utiliza **árboles de decisión** (DT) **para dividir** el espacio de características

En cada **nodo** divide el subespacio en 2 partes en base al **valor mediano en una dimensión** (la dimensión utilizada cambia en cada nodo)

Espacio dividido en regiones rectangulares que contienen un n inferior



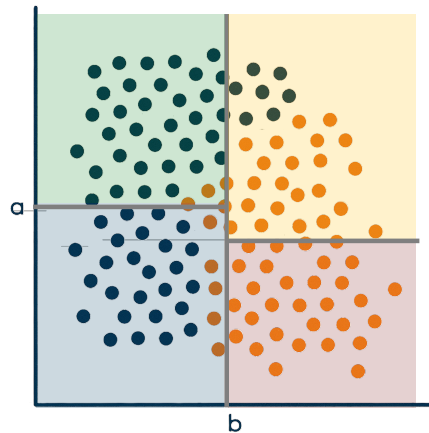
Búsqueda Informada - KD-Trees

Informed Search - KD-Trees

Utiliza **árboles de decisión** (DT) **para dividir** el espacio de características

En cada **nodo** divide el subespacio en 2 partes en base al **valor mediano en una dimensión** (la dimensión utilizada cambia en cada nodo)

¿Cuándo (o donde) funciona mal esta división del espacio de características?



Búsqueda Informada - KD-Trees

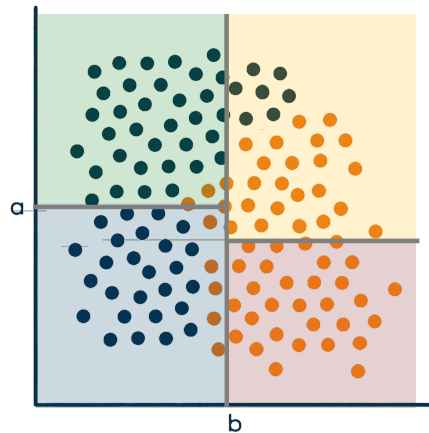
Informed Search - KD-Trees

Utiliza **árboles de decisión** (DT) **para dividir** el espacio de características

En cada **nodo** divide el subespacio en 2 partes en base al **valor mediano en una dimensión** (la dimensión utilizada cambia en cada nodo)

¿Cuando funciona mal esta división del espacio de características?

-> cuando el punto esté demasiado cerca del corte!



KD-Trees + knn

KD-Trees + knn

1. Divide el espacio de características utilizando DT
2. Cuando tienes que evaluar un punto, recorre el árbol para ver en que hoja está
3. Calcula **distancias en esa hoja**, y saca la **distancia (d)** de los vecinos más cercanos
4. Hace **Backtracking**:
 - a. Recorre el árbol hacia arriba, evaluando si la **distancia del punto con el corte de cada nodo** es menor que d
 - b. **Si es menor que d, recorre ese nodo hacia abajo**, calculando las distancias de las hojas cuyo corte estén cerca ($< d$) del punto a evaluar
5. Con todos esos vecinos “candidatos” encontramos los más cercanos

Locality-Sensitive Hashing (LSH)

**hashing: picar/dividir en trozos*

Busca una **división** (hashing) del espacio donde la **cercanía** (locality) **se mantenga**

Locality-Sensitive Hashing (LSH)

**hashing: picar/dividir en trozos*

Busca una **división** (hashing) del espacio donde la **cercanía** (locality) **se mantenga**

Pasos:

1. Generamos **k hiperplanos aleatorios** y vemos cada punto **a que lado de cada hiperplano** está, obteniendo un **vector hash para cada punto**
-> $h(x) = (1,0,0,0,1,0)$

Locality-Sensitive Hashing (LSH)

**hashing: picar/dividir en trozos*

Busca una **división** (hashing) del espacio donde la **cercanía** (locality) **se mantenga**

Pasos:

1. Generamos **k hiperplanos aleatorios** y vemos cada punto **a que lado de cada hiperplano** está, obteniendo un **vector hash para cada punto**
 -> $h(x) = (1,0,0,0,1,0)$
2. Cuando tenemos que **clasificar un punto**, calculamos a qué **bucket** pertenece: **puntos que compartan el mismo $h(x)$** .

Locality-Sensitive Hashing (LSH)

**hashing: picar/dividir en trozos*

Busca una **división** (hashing) del espacio donde la **cercanía** (locality) **se mantenga**

Pasos:

1. Generamos **k hiperplanos aleatorios** y vemos cada punto **a que lado de cada hiperplano** está, obteniendo un **vector hash para cada punto**
-> $h(x) = (1,0,0,0,1,0)$
2. Cuando tenemos que **clasificar un punto**, calculamos a qué **bucket** pertenece: **puntos que compartan el mismo $h(x)$** .

***Podemos crear varios $h(x)$ con distintos grupos de planos aleatorios**, en este caso, calculamos los buckets a los que pertenece el punto en cada espacio hash distinto y **“marcaremos” los puntos están en el mismo bucket al menos una vez.**

Locality-Sensitive Hashing (LSH)

**hashing: picar/dividir en trozos*

Busca una **división** (hashing) del espacio donde la **cercanía** (locality) **se mantenga**

Pasos:

1. Generamos **k hiperplanos aleatorios** y vemos cada punto **a que lado de cada hiperplano** está, obteniendo un **vector hash para cada punto**
 -> $h(x) = (1,0,0,0,1,0)$
2. Cuando tenemos que **clasificar un punto**, calculamos a qué **bucket** pertenece: **puntos que compartan el mismo $h(x)$** .

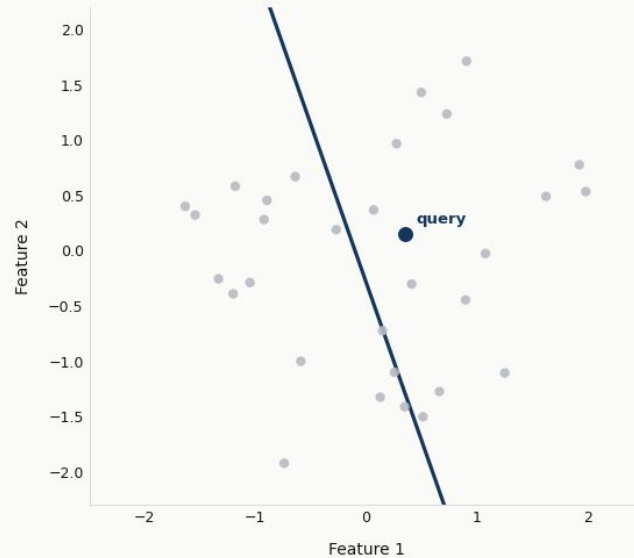
***Podemos crear varios $h(x)$ con distintos grupos de planos aleatorios**, en este caso, calculamos los buckets a los que pertenece el punto en cada espacio hash distinto y **“marcaremos” los puntos están en el mismo bucket al menos una vez.**
3. **Sobre esos puntos**, calculamos la distancia (en el **espacio original de features**, no en el hash!) y **aplicamos knn**

Locality-Sensitive Hashing (LSH)

Ejemplo

01 RANDOM HYPERPLANES

Each hyperplane splits the feature space into 0 / 1



Locality-Sensitive Hashing (LSH)

¿Qué hiperparámetros tiene?

Pasos:

1. Generamos **k hiperplanos aleatorios** y vemos cada punto **a que lado de cada hiperplano** está, obteniendo un **vector hash para cada punto**
 -> $h(x) = (1,0,0,0,1,0)$
2. Cuando tenemos que **clasificar un punto**, calculamos a qué **bucket** pertenece: **puntos que compartan el mismo $h(x)$** .
 *Podemos crear varios $h(x)$ con distintos grupos de planos aleatorios, en este caso, calculamos los buckets a los que pertenece el punto en cada espacio hash distinto y “**marcaremos**” los puntos están en el mismo bucket al menos una vez.
3. **Sobre esos puntos**, calculamos la distancia (en el **espacio original de features**, no en el hash!) y aplicamos knn

Locality-Sensitive Hashing (LSH)

¿Qué hiperparámetros tiene?

3 hiperparámetros!

Pasos:

1. Generamos **k hiperplanos aleatorios** y vemos cada punto a que lado de cada hiperplano está, obteniendo un vector hash para cada punto
-> $h(x) = (1,0,0,0,1,0)$
2. Cuando tenemos que clasificar un punto, calculamos a qué bucket pertenece: puntos que compartan el mismo $h(x)$.

*Podemos crear varios $h(x)$ con **distintos grupos de planos aleatorios**, en este caso, calculamos los buckets a los que pertenece el punto en cada espacio hash distinto y “marcaremos” los puntos están en el mismo bucket al menos una vez.
3. Sobre esos puntos, calculamos la distancia (en el espacio original de features, no en el hash!) y aplicamos **knn**

Locality-Sensitive Hashing (LSH)

Hiperparámetros

**hashing: picar/dividir en trozos*

Busca una **división** (hashing) del espacio donde la **cercanía** (locality) **se mantenga**

Tres hiperparámetros:

- Número de **hiperplanos aleatorios (k)** (Longitud del hash vector)
-> cuanto mayor, más pequeños los buckets y más rápido será el método pero menos preciso
- Cantidad de **buckets (L)** (proyecciones hash distintas / grupos de k hiperplanos)
-> mayor cantidad, menos riesgo de no evaluar los mejores vecinos, pero más coste computacional
- Cuantos **vecinos más cercanos** (knn típico)



Caso real de ANN: ANNOY - Spotify

Real use case of ANN: Approximate Nearest Neighbors Oh Yeah (ANNOY)

Se crea para un contexto muy concreto: recomendar canciones similares de una base de datos de millones de canciones a millones de usuarios y en tiempo real

Implicaciones:

- Miles de búsquedas por segundo
- Construcción razonablemente rápida (se actualiza frecuentemente)
- Indexar nuevos ítems sin rehacer todo (cada nueva canción/usuario reentrenar todo?!?)

Knn como regresor

Carlos Bibián Nogueras, PhD

cbibian@itba.edu.ar |

Instituto Tecnológico de Buenos Aires

KNN como regresor

KNN as a regressor

Knn como clasificador: Asigna la clase más votada de los k vecinos más cercanos

Knn como regresor: el valor $y(x)$ se calcula como el promedio del valor de y de los k vecinos más cercanos

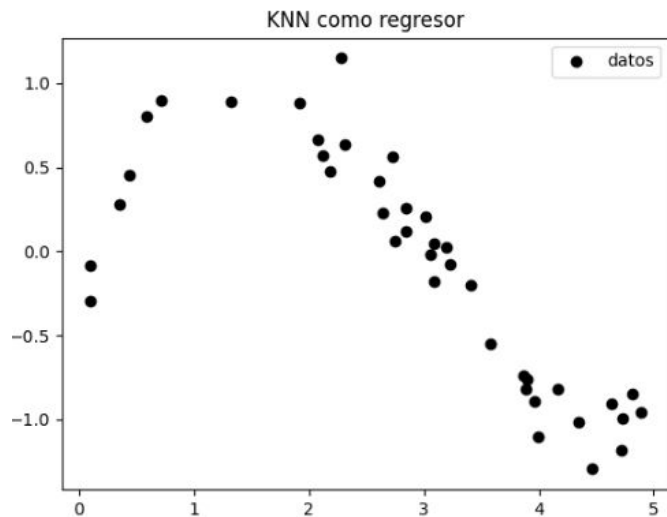
$$\hat{y}(x) = \frac{1}{k} \sum_{i \in N_k(x)} y_i$$

weighted-knn como regresor: el valor $y(x)$ se calcula como el promedio ponderado con el inverso de la distancia del valor de y de los k vecinos más cercanos

KNN como regresor

KNN as a regressor

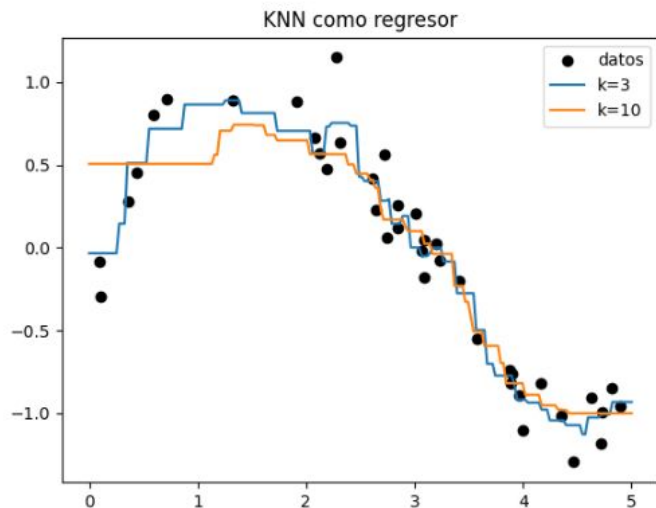
Knn como regresor: el valor $y(x)$ se calcula como el promedio del valor de y de los k vecinos más cercanos



KNN como regresor

KNN as a regressor

Knn como regresor: el valor $y(x)$ se calcula como el promedio del valor de y de los k vecinos más cercanos



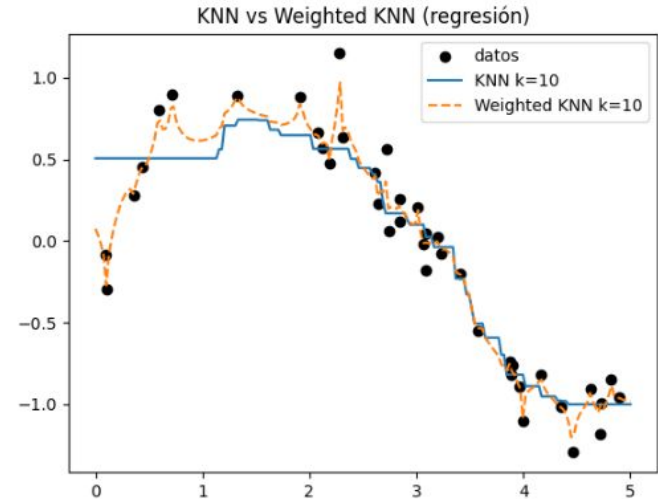
Un **k alto**, da una **regresión más suavizada** pero atenúa más los “máximos” de la función

Igual que en clasificación, un k pequeño más tendencia al overfitting

KNN como regresor

KNN as a regressor

- Modelo no paramétrico (no asume nada sobre el modelo)
- No aprende una función global
- Muy sensible a escala y ruido
- Puede modelar relaciones muy complejas
- Igual que en clasificación:
 - Es muy costoso computacionalmente
 - Escala mal con datasets grandes y muchas dimensiones (features)



KNN como regresor

KNN as a regressor

Mejora de eficiencia:

Podemos también usar KD-Trees para mejorar la eficiencia

- Con muchas dimensiones (>20) no es tan efectivo ya que las distancias son cada vez menos informativas)

Podemos usar también ANN, pero en este caso, al ser un cálculo aproximado de vecinos, la regresión se ve más afectada que la clasificación (calculamos un valor de y , no una clase)

DT como regresor

Carlos Bibián Nogueras, PhD

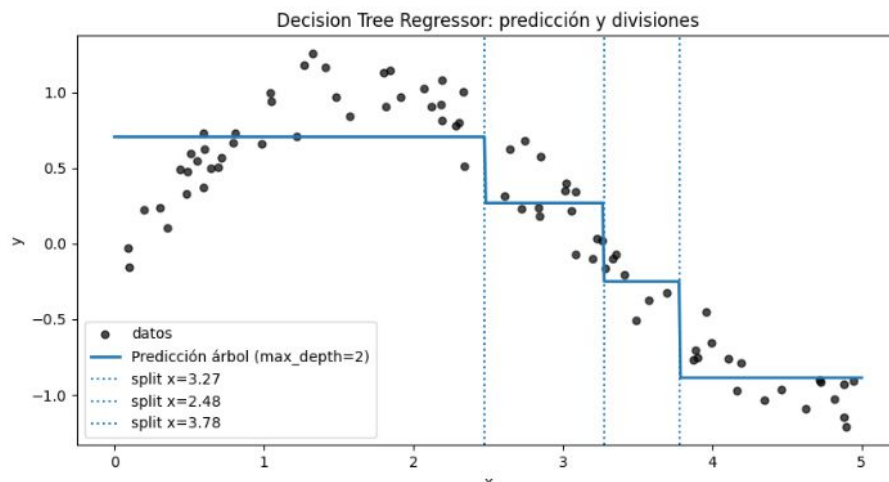
cbibian@itba.edu.ar |

Instituto Tecnológico de Buenos Aires

DT (y RF) como regresor

DT (and RF) as a regressor

Divide el espacio en hojas, y la predicción de y para un valor de x es la y promedio de las x de esa hoja

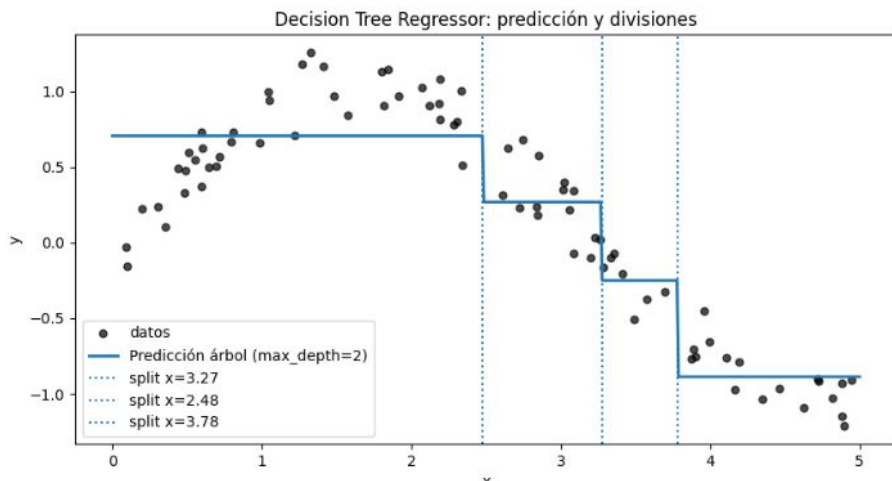


DT (y RF) como regresor

DT (and RF) as a regressor

Divide el espacio en hojas, y la predicción de y para un valor de x es la y promedio de las x de esa hoja

max_depth, me permite más divisiones del espacio de feats, salida más ruidosa/sensible a los datos



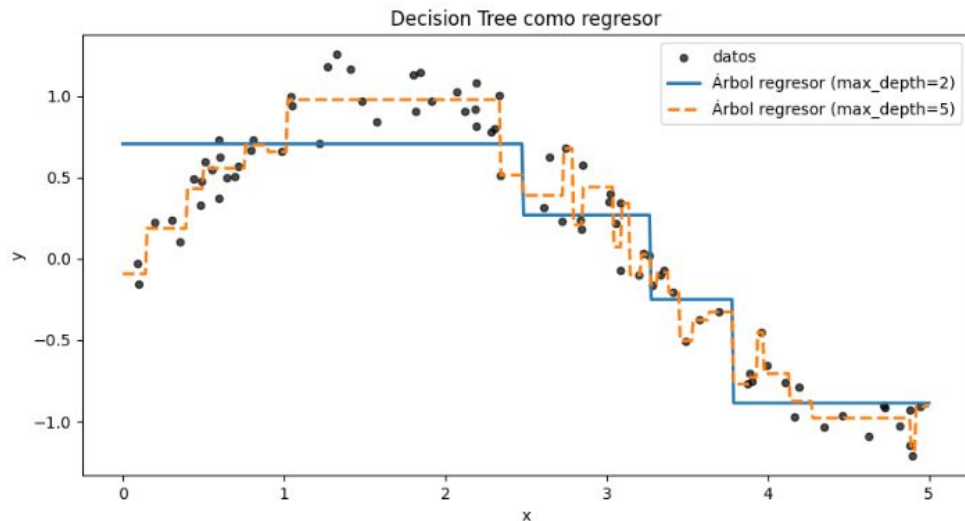
DT (y RF) como regresor

DT (and RF) as a regressor

Ventajas e inconvenientes

Como en clasificación:

- puede capturar geometrias complejas
- muy interpretable
- propenso al overfitting
- inestable



Comparación DT y RF como regresores

DT vs RF regressors

Divide el espacio en hojas, y la predicción de y para un valor de x es la y promedio de las x de esa hoja

Como en clasificación, RF:

- puede capturar geometrías complejas
- menos interpretable
- menos propenso al overfitting
- robusto

